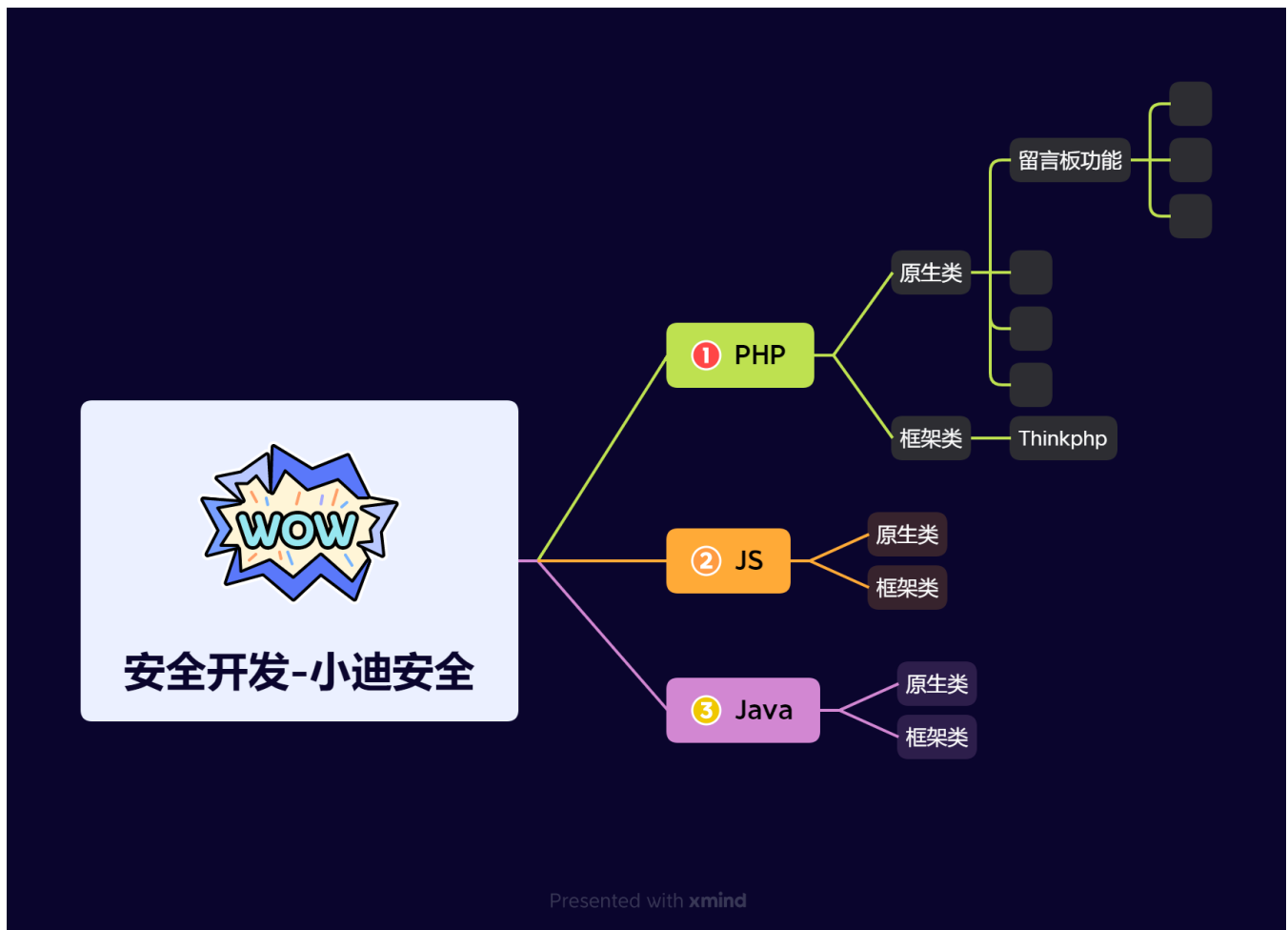


Day34 安全开发-JavaEE应用&反射机制&攻击链&类对象&成员变量方法&构造方法



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

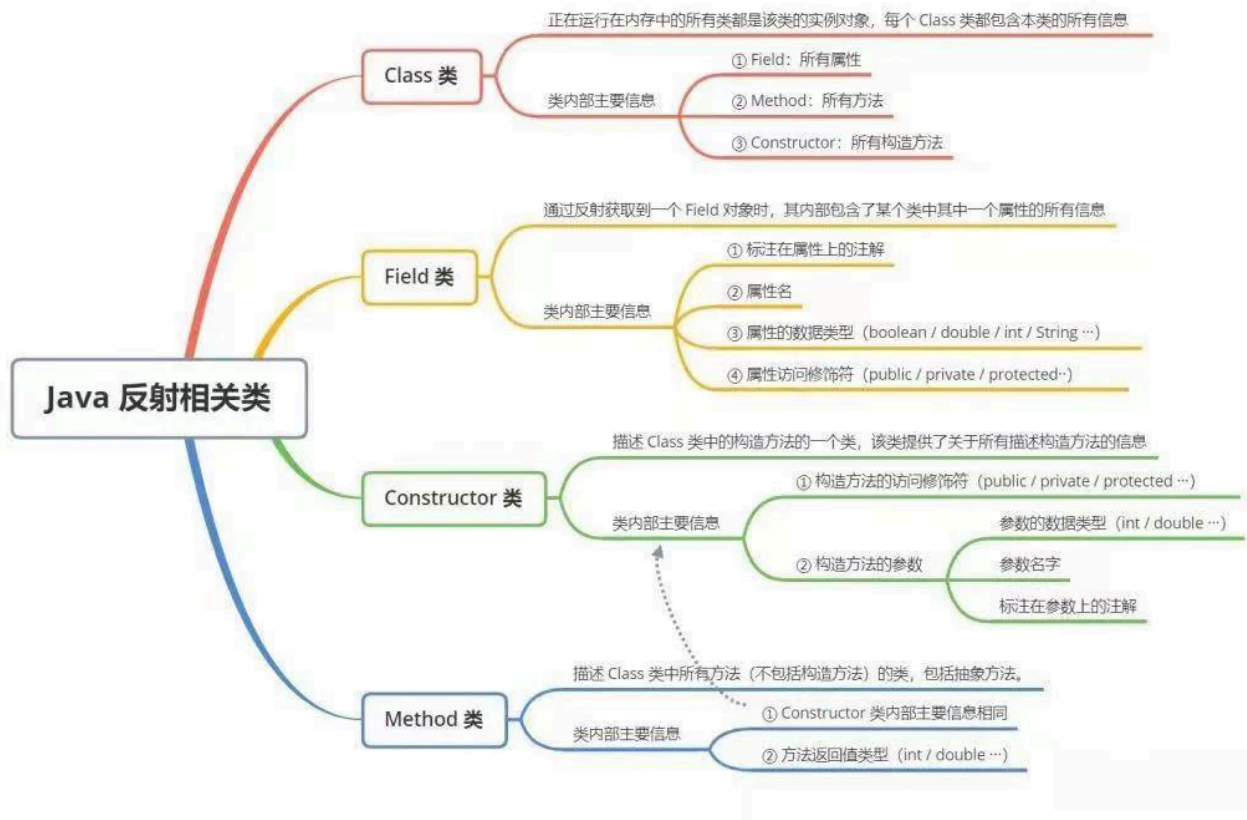
- 1、三方库-JQuery-使用&安全
 - 2、打包器-Webpack-使用&安全
-

- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis
-

- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器
-

- 1、JavaEE-反射机制-开发和安全应用场景
- 2、JavaEE-反射机制-类&成员变量&方法&构造方法操作
- 3、JavaEE-反射机制-安全应用&反射执行&反序列化链相关

2.演示案例



利用反射获取成员变量

Class类中用于获取成员变量的方法

Field[] getFields(): 返回所有公共成员变量对象的数组

Field[] getDeclaredFields(): 返回所有成员变量对象的数组

Field getField(String name): 返回单个公共成员变量对象

Field getDeclaredField(String name): 返回单个成员变量对象

Field类中用于创建对象的方法

void set(Object obj, Object value): 赋值

Object get(Object obj) 获取值。

利用反射获取构造方法

Class类中用于获取构造方法的方法

Constructor<?>[] getConstructors(): 返回所有公共构造方法对象的数组

Constructor<?>[] getDeclaredConstructors(): 返回所有构造方法对象的数组

Constructor<T> getConstructor(Class<?>... parameterTypes): 返回单个公共构造方法对象

Constructor<T> getDeclaredConstructor(Class<?>... parameterTypes): 返回单个构造方法对象

Constructor类中用于创建对象的方法

T newInstance(Object... initargs): 根据指定的构造方法创建对象

setAccessible(boolean flag): 设置为true,表示取消访问检查

利用反射获取成员方法

Class类中用于获取成员方法的方法

Method[] getMethods(): 返回所有公共成员方法对象的数组，包括继承的

Method[] getDeclaredMethods(): 返回所有成员方法对象的数组，不包括继承的

Method getMethod(String name, Class<?>... parameterTypes) : 返回单个公共成员方法对象

Method getDeclaredMethod(String name, Class<?>... parameterTypes): 返回单个成员方法对象

Method类中用于创建对象的方法

Object invoke(Object obj, Object... args): 运行方法

参数一：用obj对象调用该方法

参数二：调用方法的传递的参数（如果没有就不写）

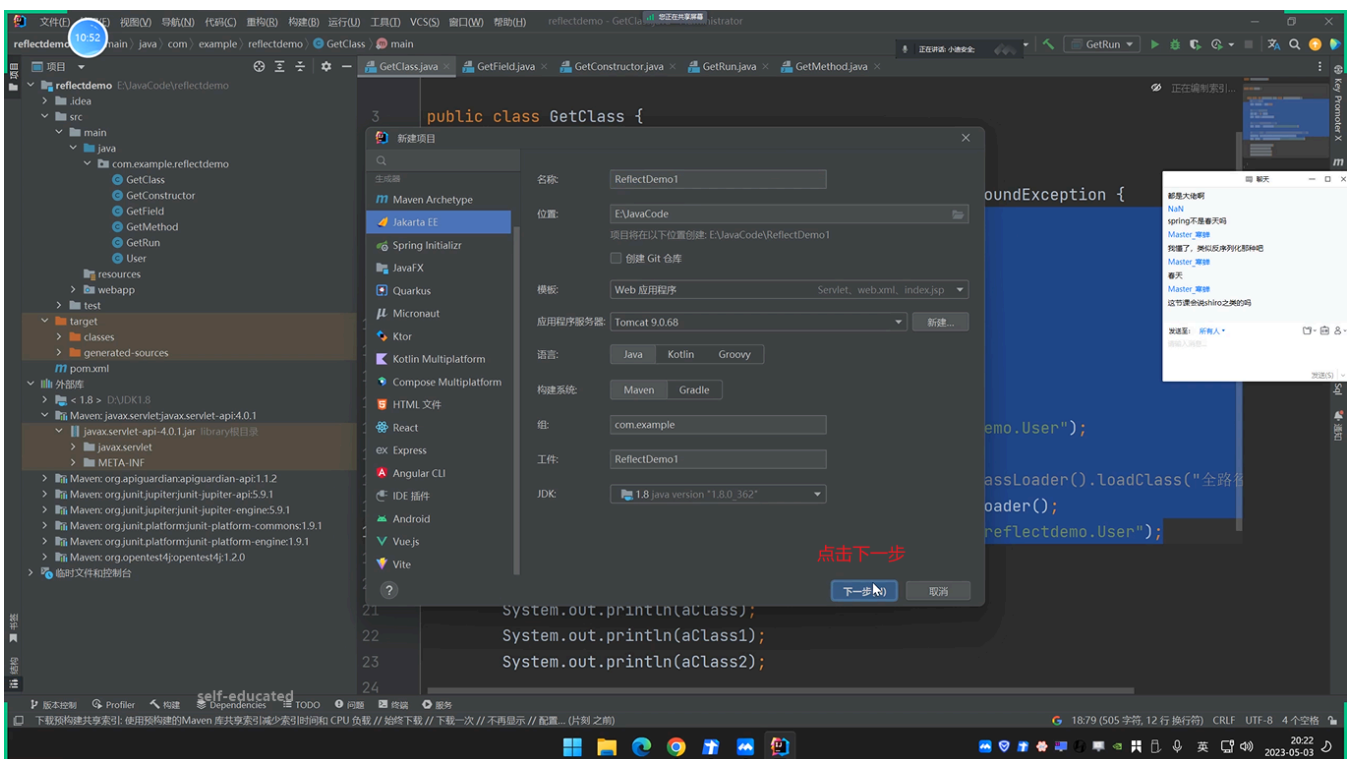
返回值：方法的返回值（如果没有就不写）

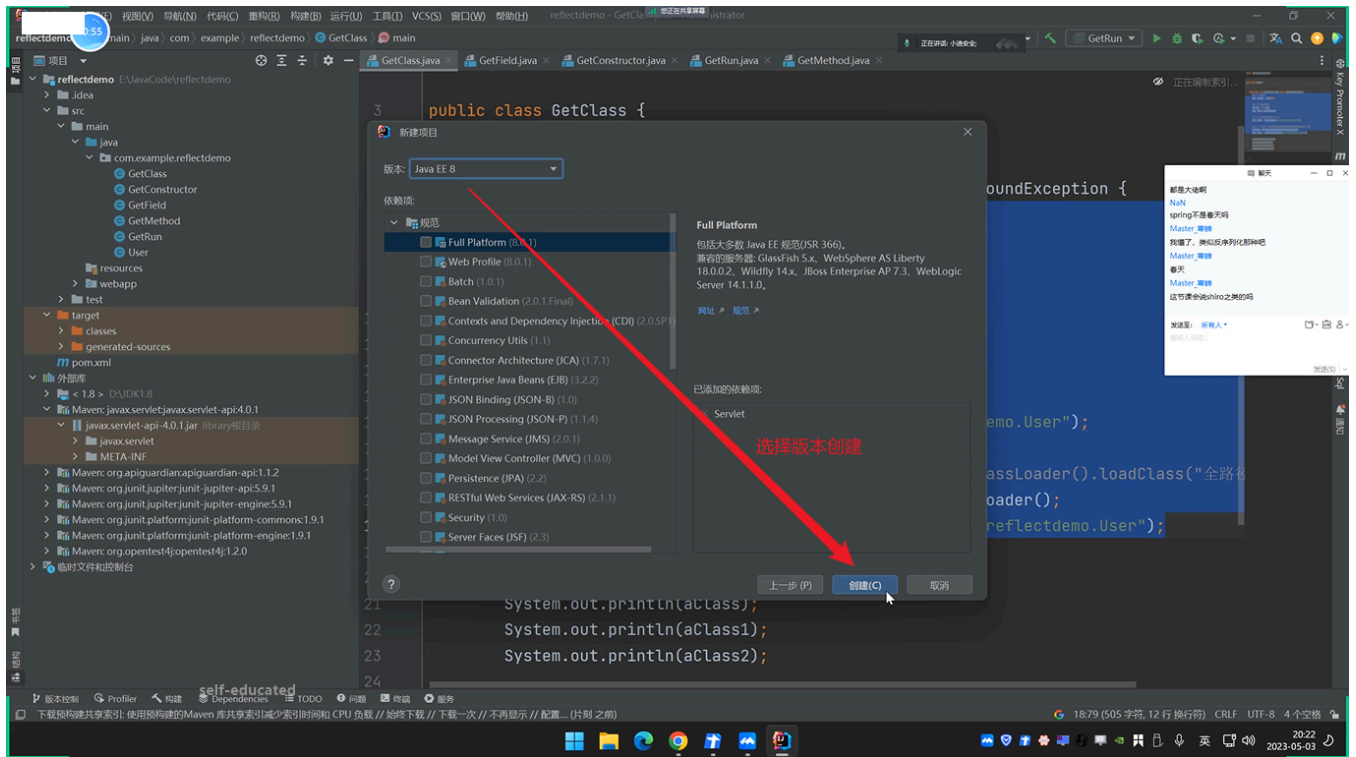
```
1  1、什么是Java反射
2  参考: https://xz.aliyun.com/t/9117
3  Java提供了一套反射API，该API由Class类与java.lang.reflect类库组成。
4  该类库包含了Field、Method、Constructor等类。
5  对成员变量，成员方法和构造方法的信息进行的编程操作可以理解为反射机制。
6
7  2、为什么要用到反射
8  参考: https://xz.aliyun.com/t/9117
9  其实从官方定义中就能找到其存在的价值，在运行时获得程序或程序集中每一个类型的成员和成员的信息，从而动态
   的创建、修改、调用、获取其属性，而不需要事先知道运行的对象是谁。划重点：在运行时而不是编译时。（不改变
   原有代码逻辑，自行运行的时候动态创建和编译即可）
10
11 3、反射机制应用
12 开发应用场景：
13 Spring框架的IOC基于反射创建对象和设置依赖属性。
14 SpringMVC的请求调用对应方法，也是通过反射。
15 JDBC的Class.forName(String className)方法，也是使用反射。
16
17 安全应用场景：
18 构造利用链，触发命令执行
19 反序列化中的利用链构造
20 动态获取或执行任意类中的属性或方法
21 动态代理的底层原理是反射技术
22 rmi反序列化也涉及到反射操作
```

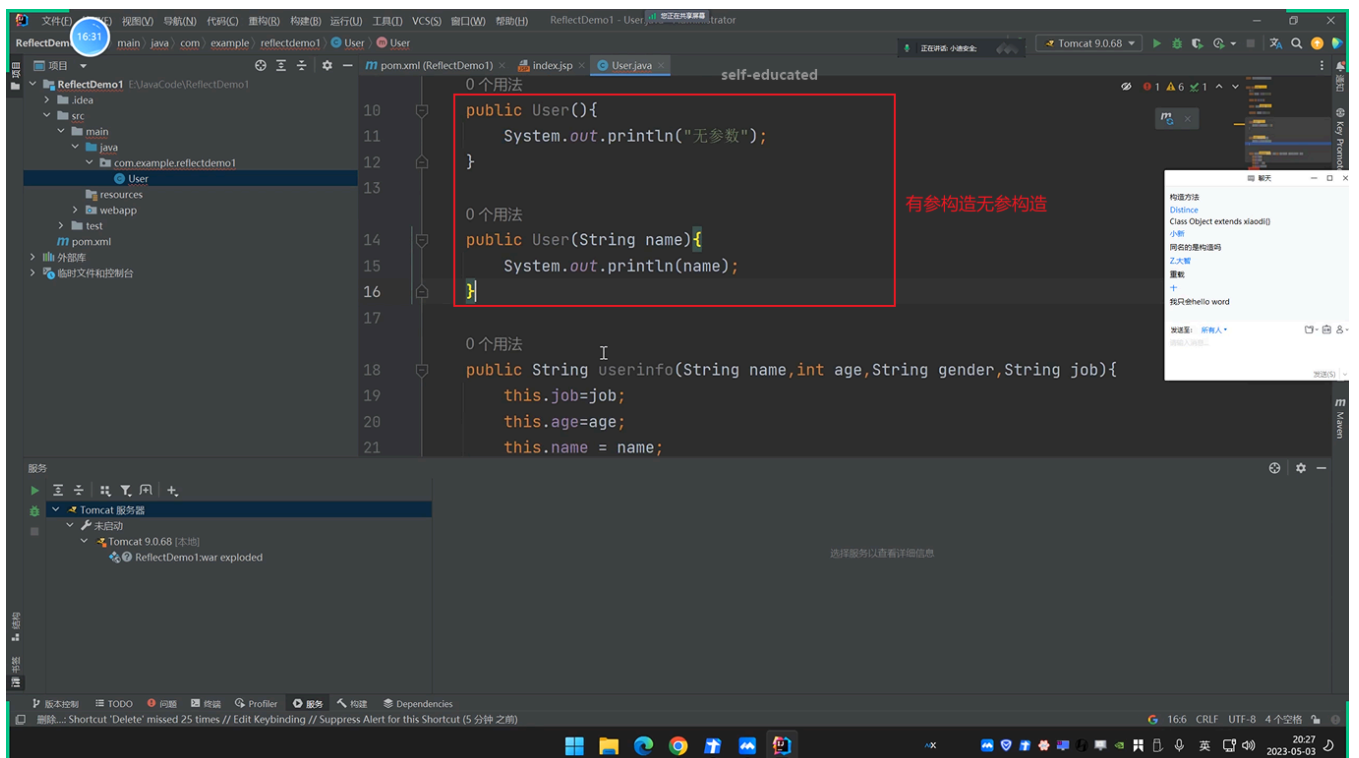
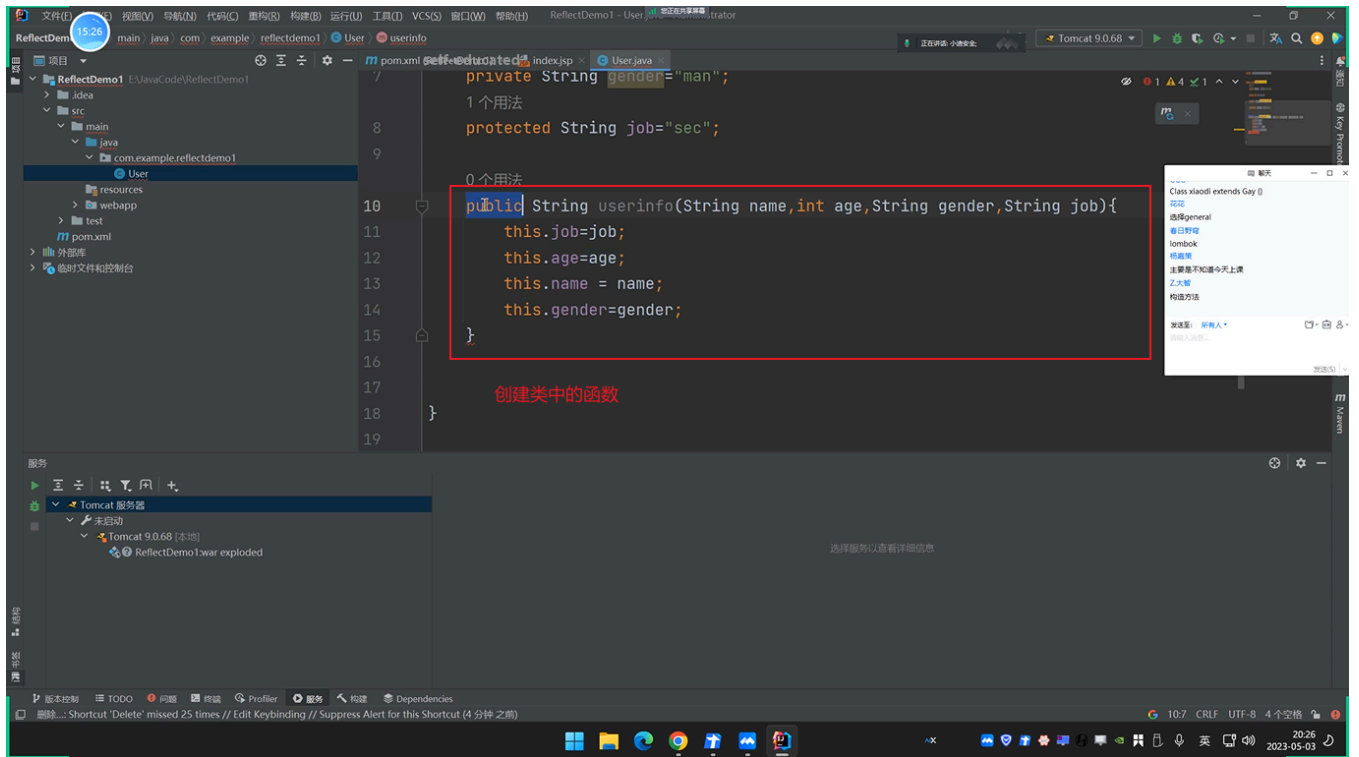
2.1 Java-反射-Class对象类获取

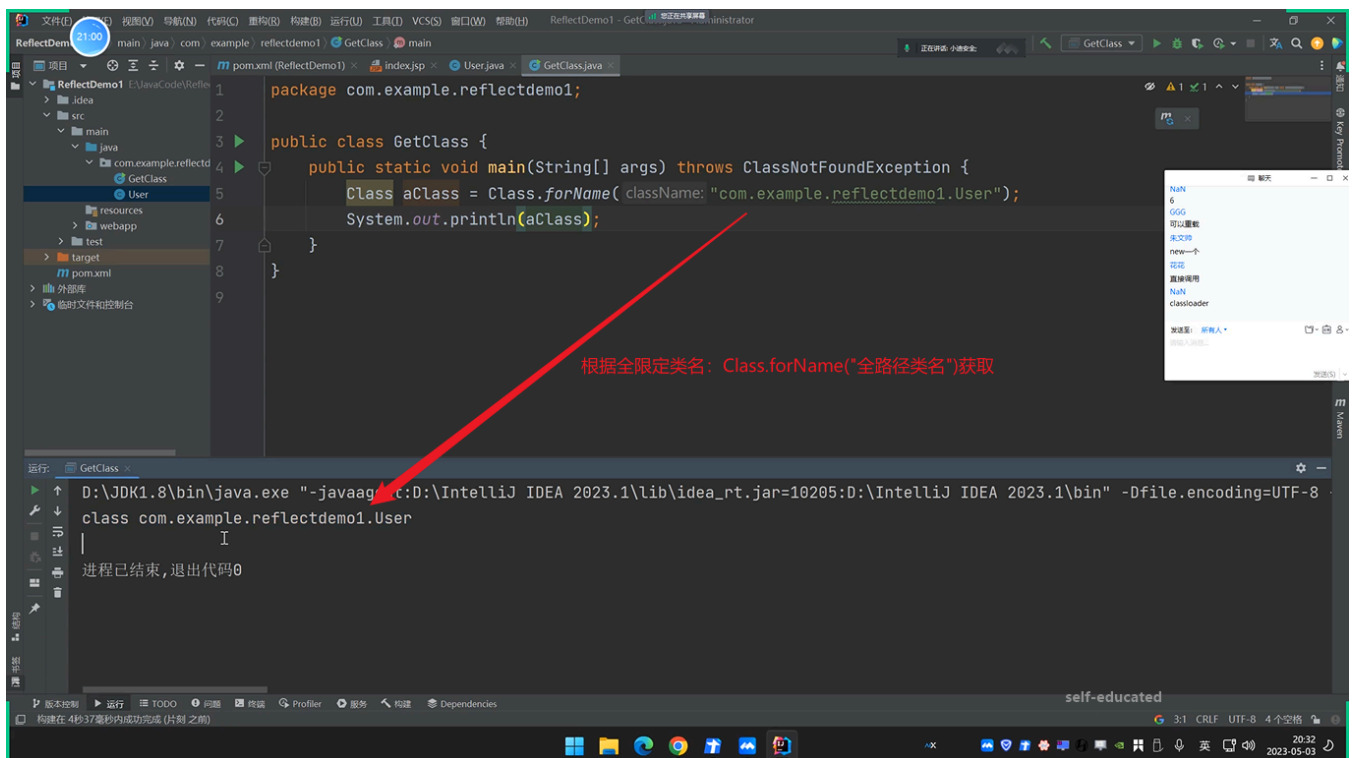
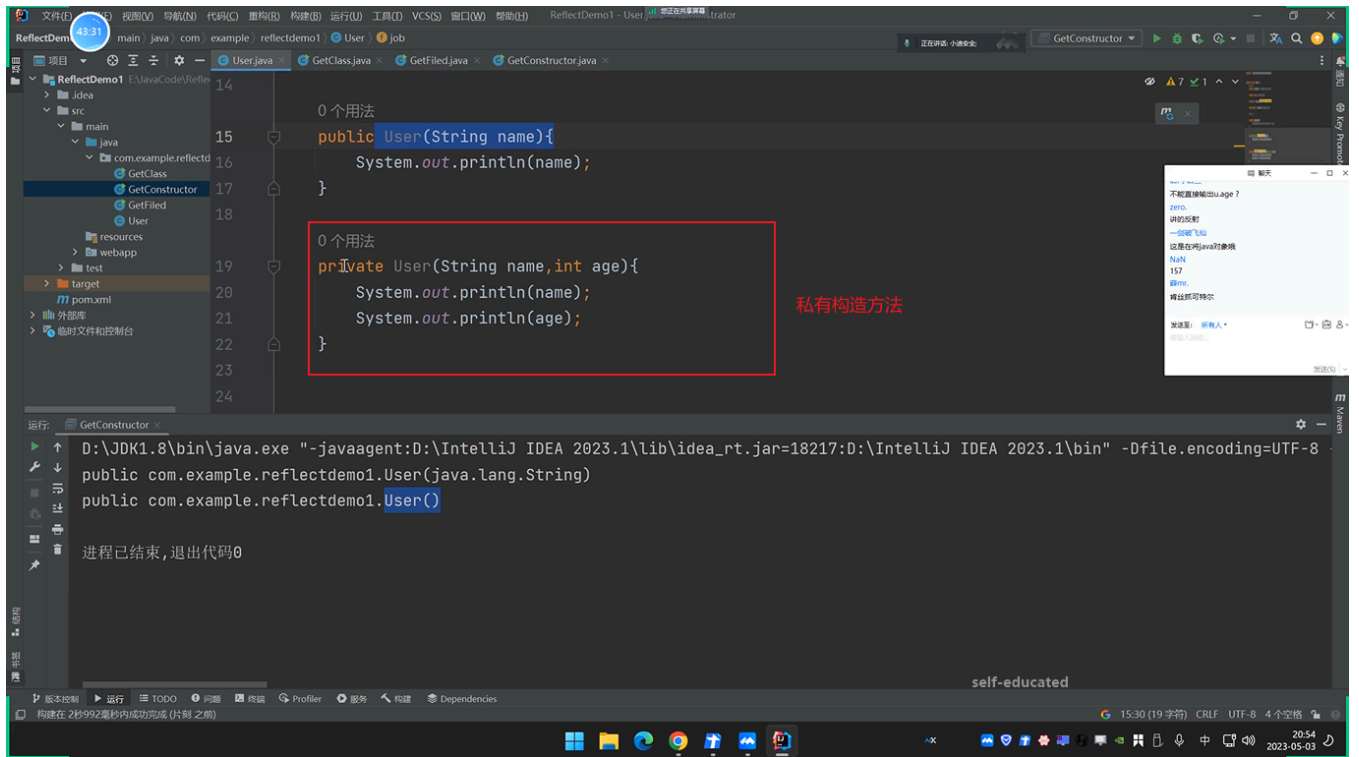
```
1 //1、根据类名：类名.class
2 Class userClass = User.class;
3 //2、根据对象：对象.getClass()
4 User user = new User();
5 Class aClass = user.getClass();
6 //3、根据全限定类名：Class.forName("全路径类名")
7 Class aClass1 = Class.forName("com.example.reflectdemo.User");
8 //4、通过类加载器获得Class对象：
9 //ClassLoader.getSystemClassLoader().loadClass("全路径类名");
10 ClassLoader clsload=ClassLoader.getSystemClassLoader();
11 Class aClass2 = clsload.loadClass("com.example.reflectdemo.User");
```

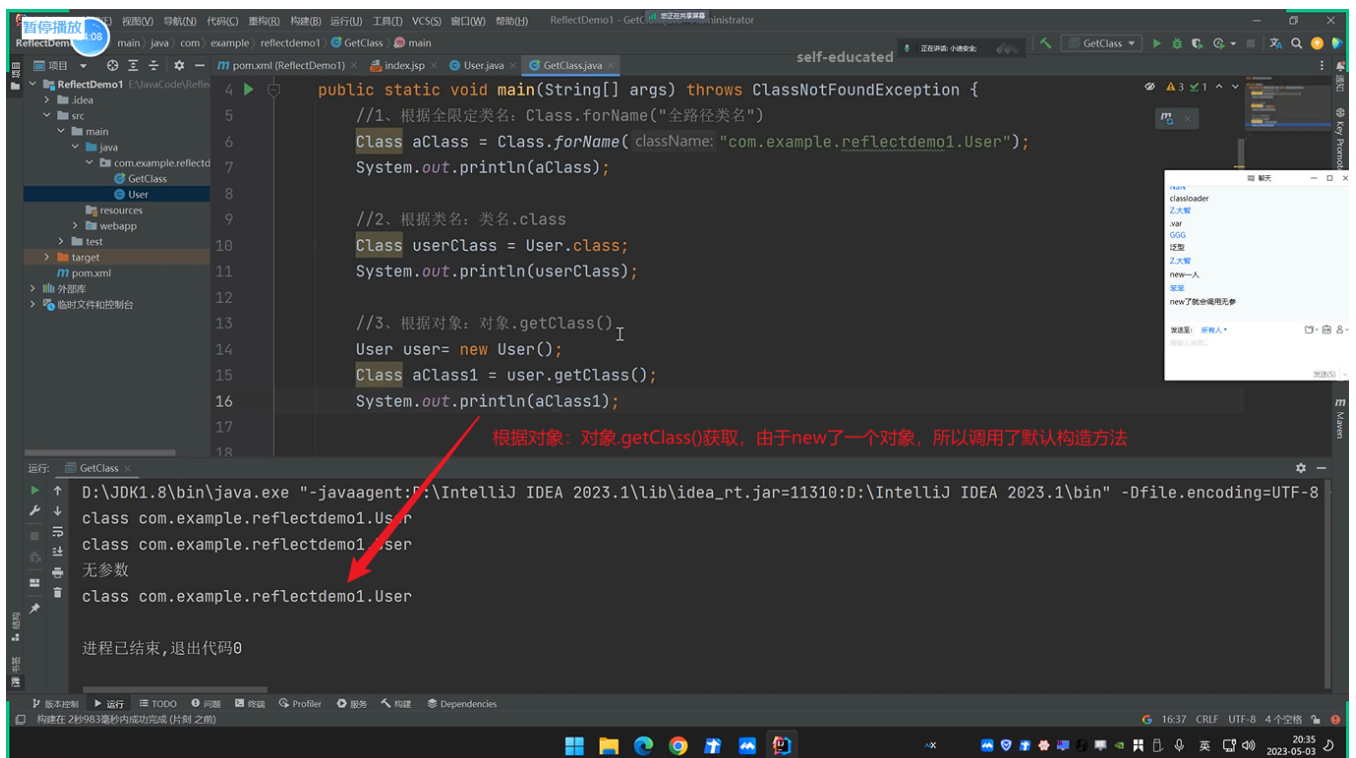
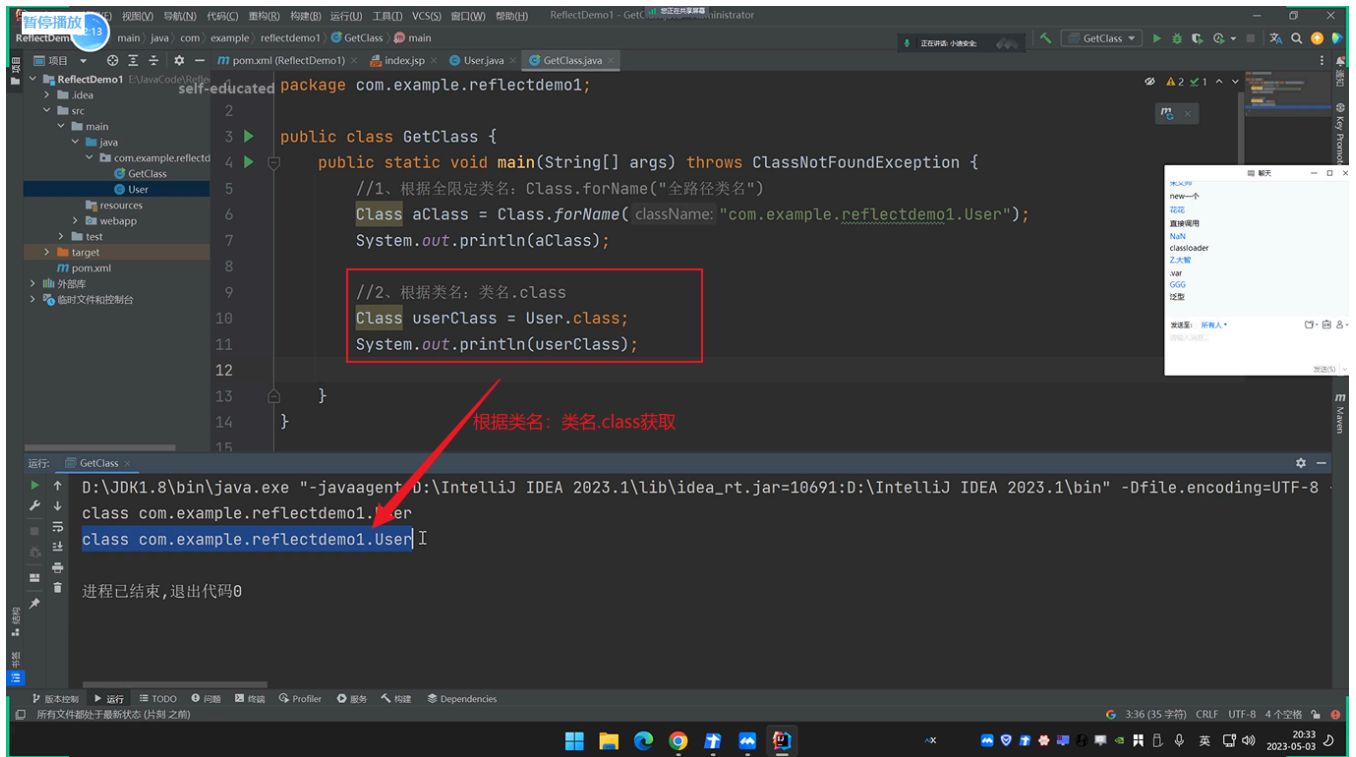
(1) 创建项目：

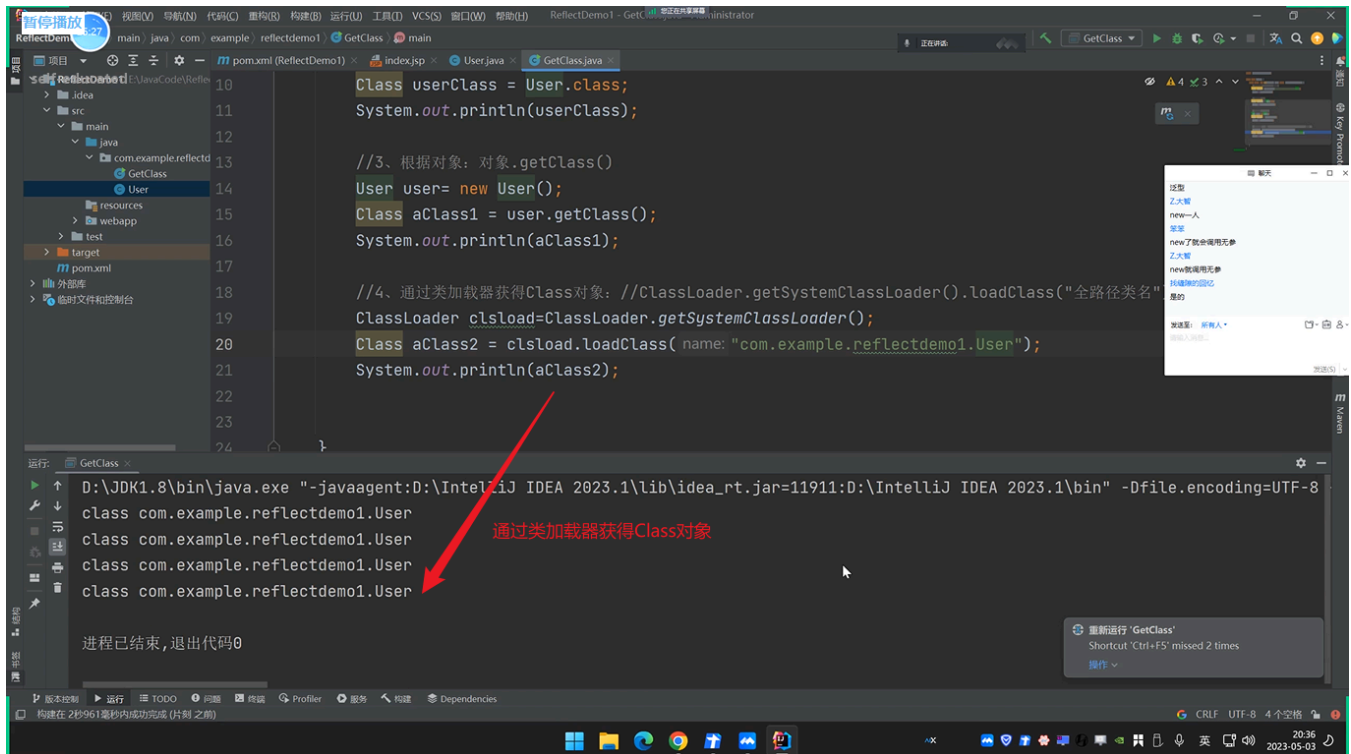








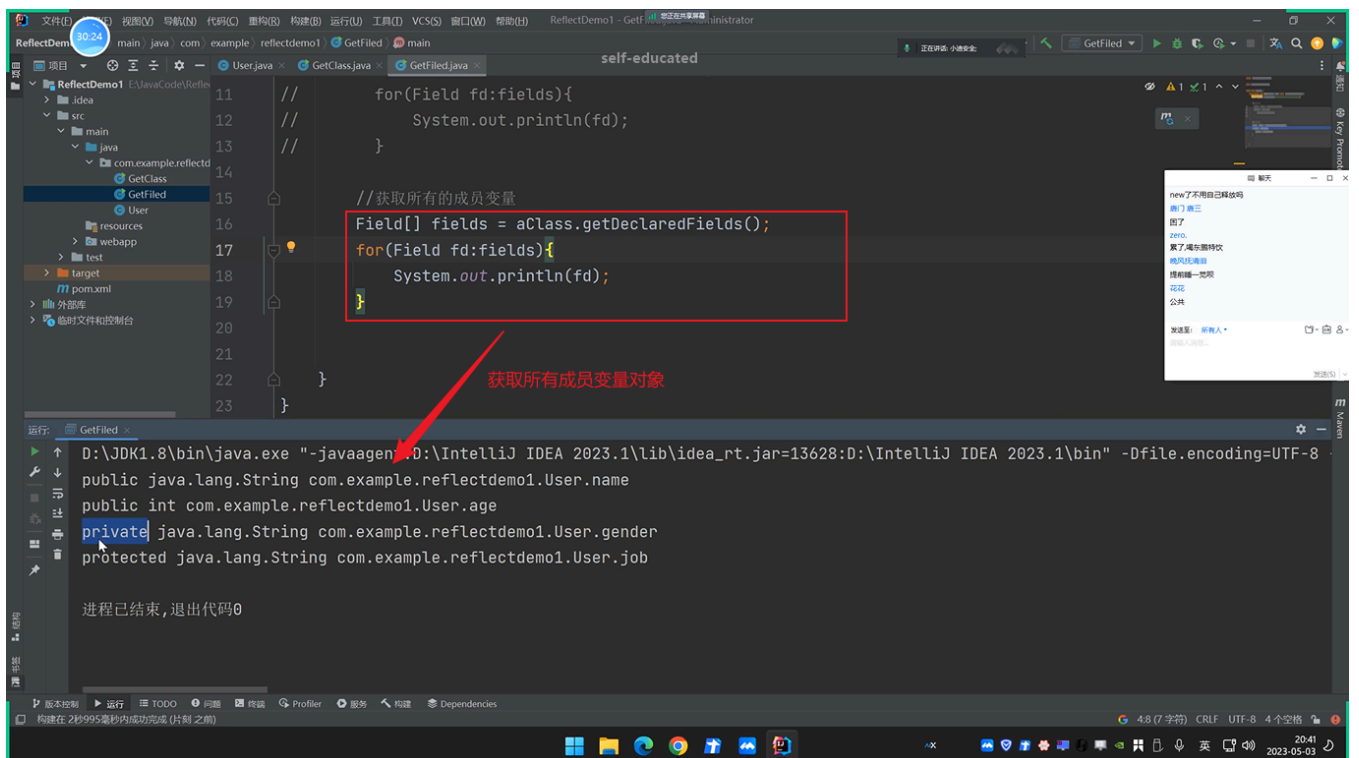
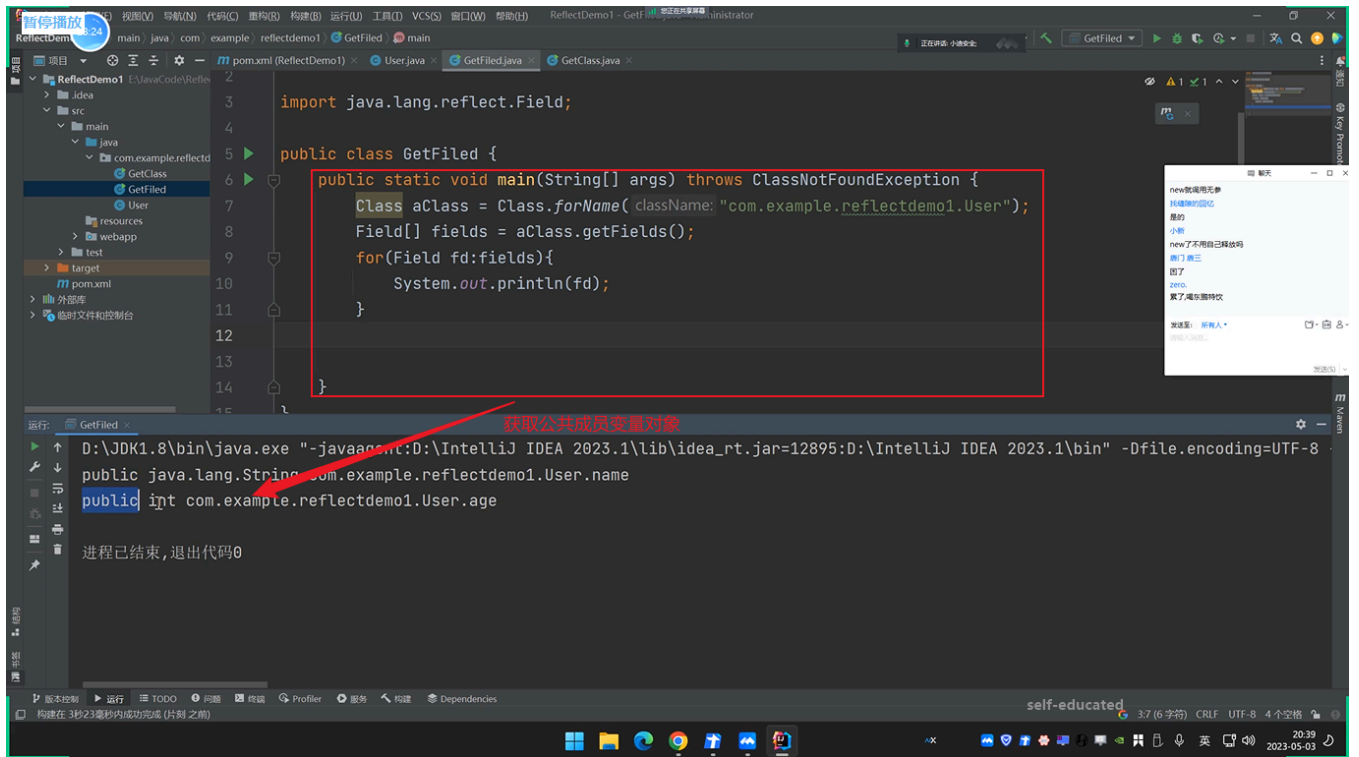


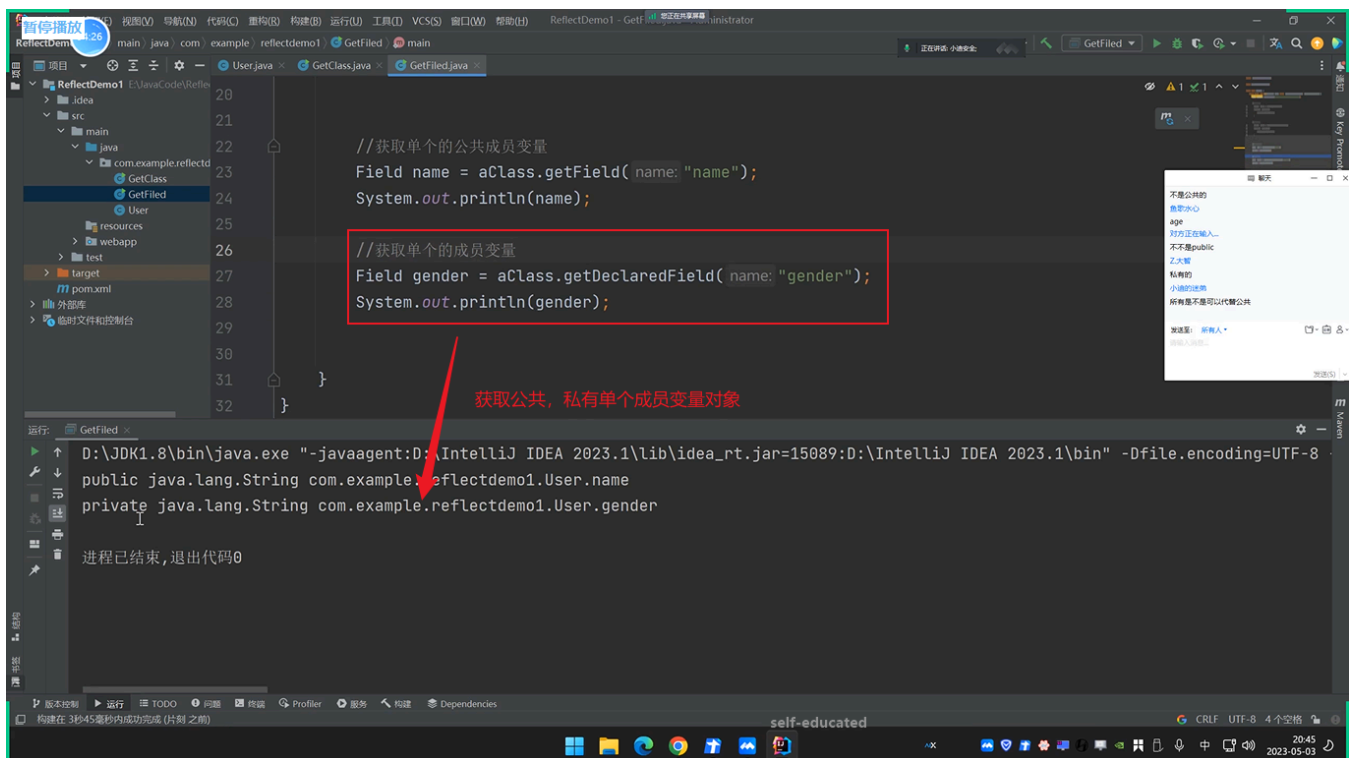
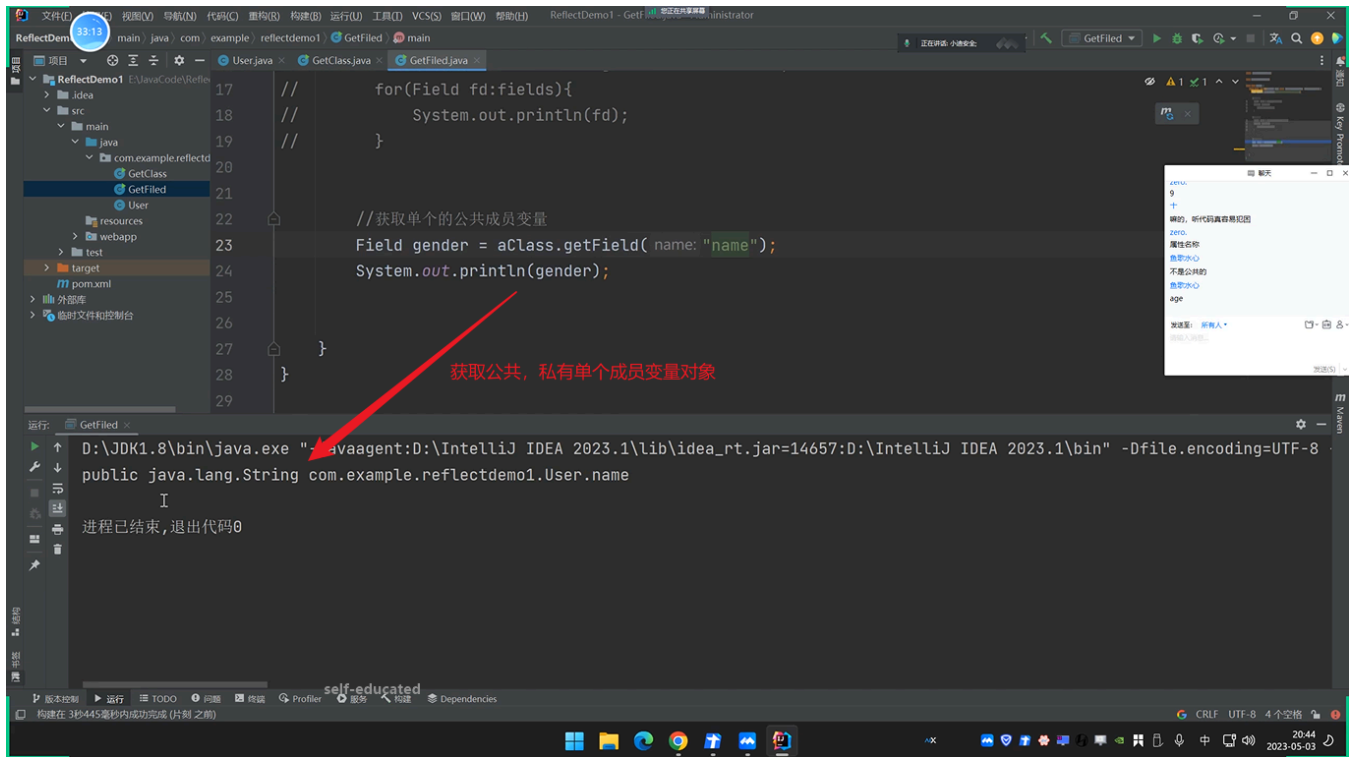


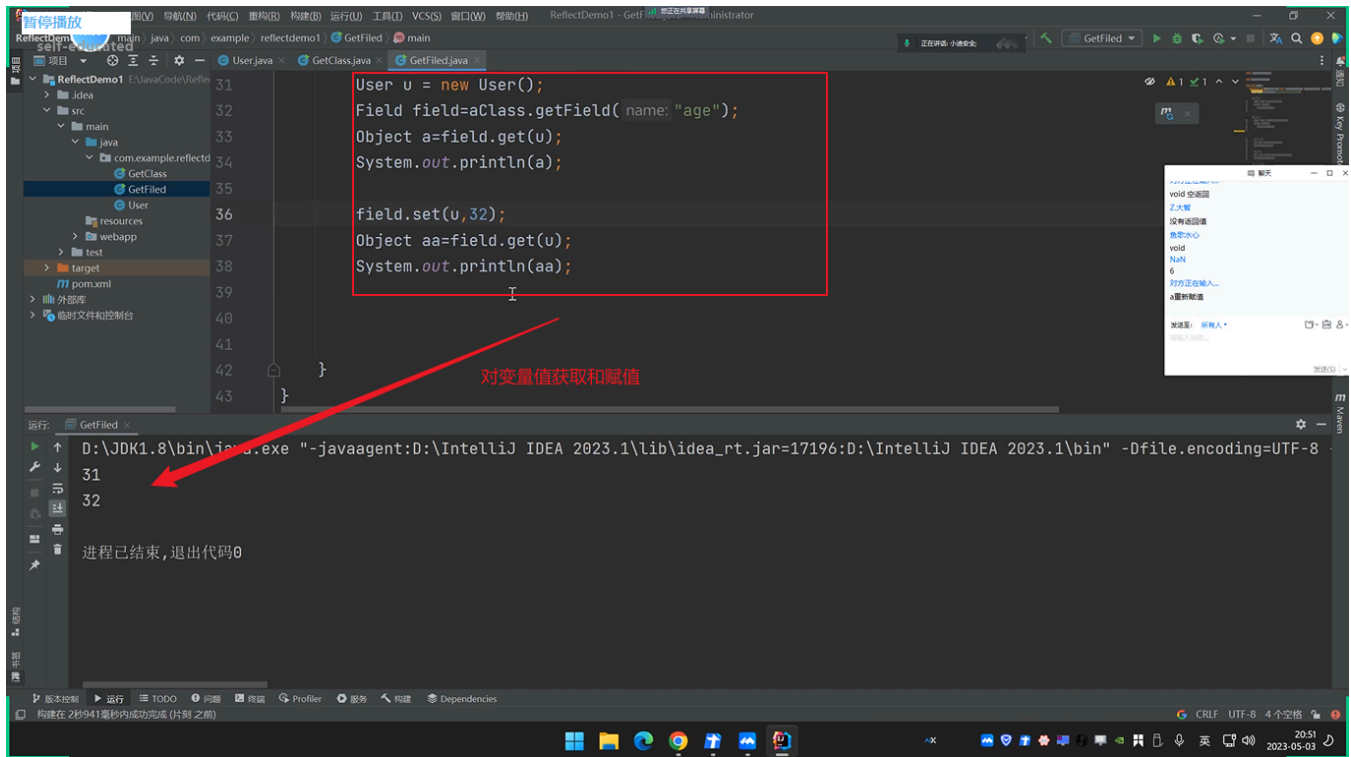
2.2 Java-反射-Field成员变量类获取

```
1 //Class aClass = Class.forName("com.example.reflectdemo.User");
2 //获取公共成员变量对象
3 //    Field[] fields=aClass.getFields();
4 //    for(Field f:fields){
5 //        system.out.println(f);
6 //    }
7
8 //获取所有成员变量对象
9 //    Field[] fields=aClass.getDeclaredFields();
10 //    for(Field f:fields){
11 //        system.out.println(f);
12 //    }
13
14 //获取公共，私有单个成员变量对象
15 //    Field field=aClass.getField("age");
16 //    Field field=aClass.getDeclaredField("gender");
17 //    System.out.println(field);
18
19 //对变量值获取和赋值
20 //    User u = new User();
21 //    Field field=aClass.getField("age");
22 //    field.set(u,30);
23 //    Object a=field.get(u);
24 //    System.out.println(a);
```

(1) Field成员变量类获取:







2.3 Java-反射-Constructor构造方法类获取

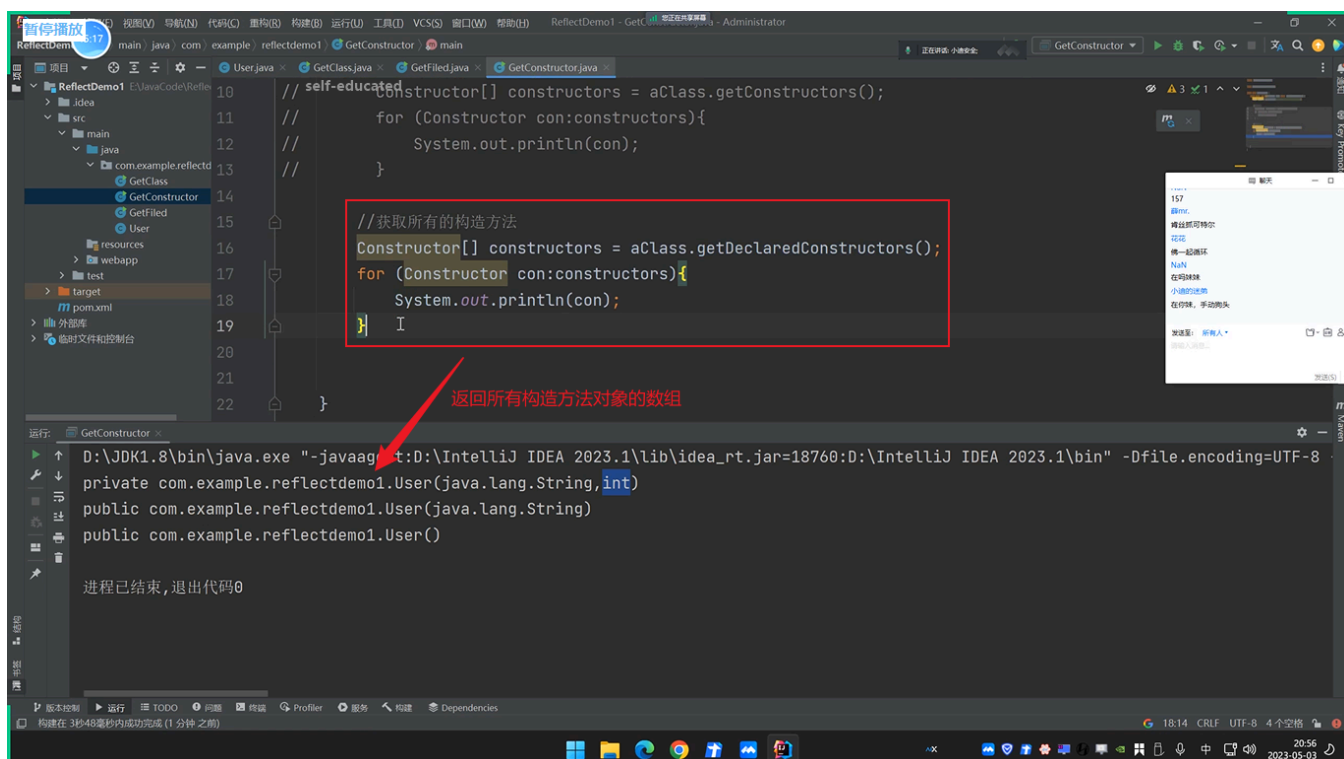
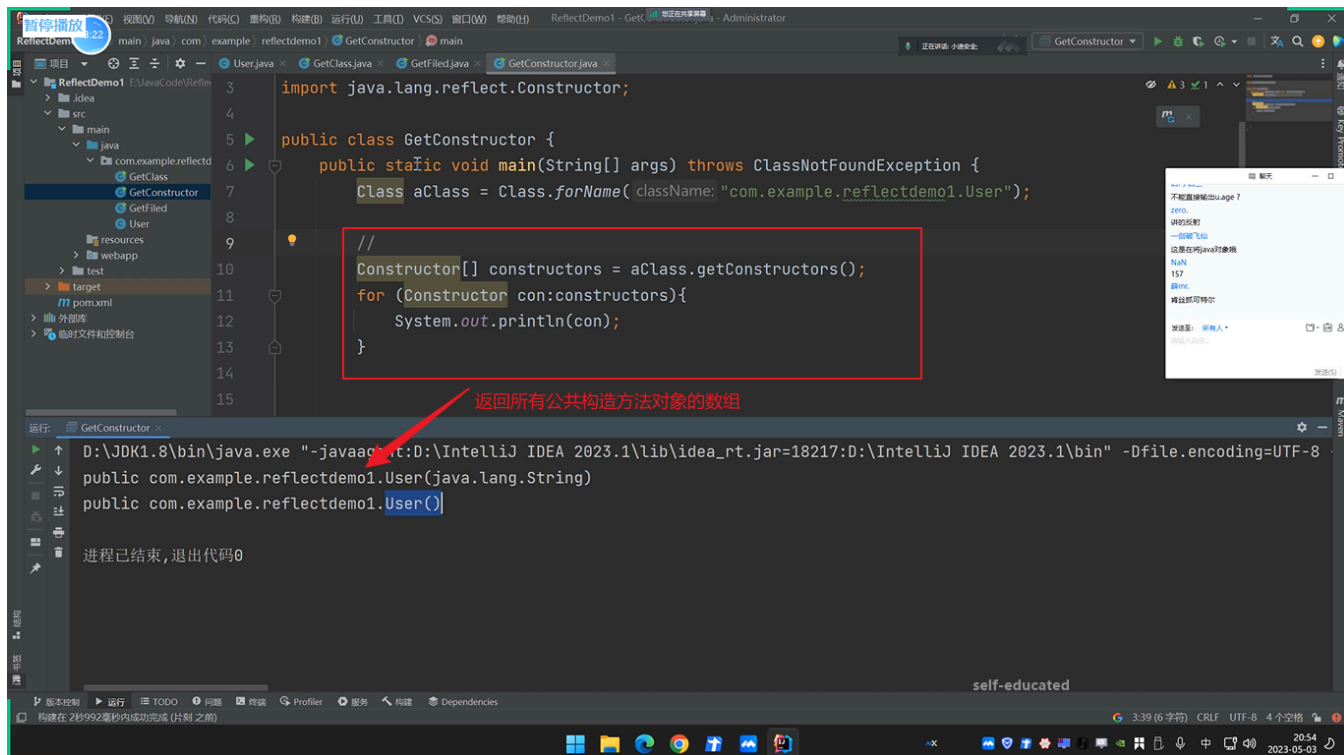
```
1  //Class aClass = Class.forName("com.example.reflectedemo.User");
2  //返回所有公共构造方法对象的数组
3  //    Constructor[] constructors = aClass.getConstructors();
4  //    for(Constructor con:constructors){
5  //        System.out.println(con);
6  //    }
7
8  //返回所有构造方法对象的数组
9  //    Constructor[] constructors = aClass.getDeclaredConstructors();
10 //    for(Constructor con:constructors){
11 //        System.out.println(con);
12 //    }
13
14 //返回单个公共构造方法对象
15 //    Constructor con1=aClass.getConstructor();
16 //    Constructor con1=aClass.getConstructor(String.class);
17 //    System.out.println(con1);
18
19 //返回单个构造方法对象
20 //    Constructor con2=aClass.getDeclaredConstructor(int.class);
21 //Constructor con2=aClass.getDeclaredConstructor(String.class,int.class,
22 //String.class);
23 //    System.out.println(con2);
24 //    Constructor con2=aClass.getDeclaredConstructor(int.class);
25 //    con2.setAccessible(true);
```

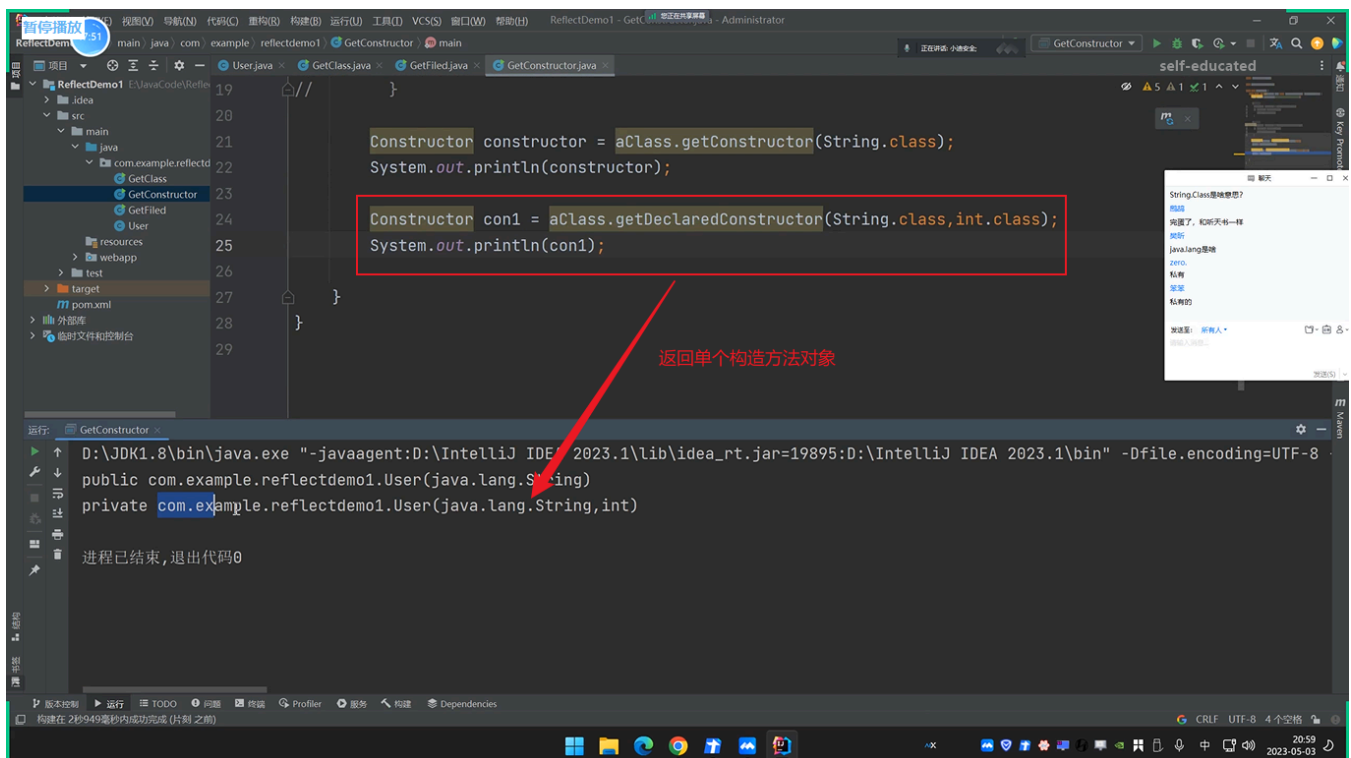
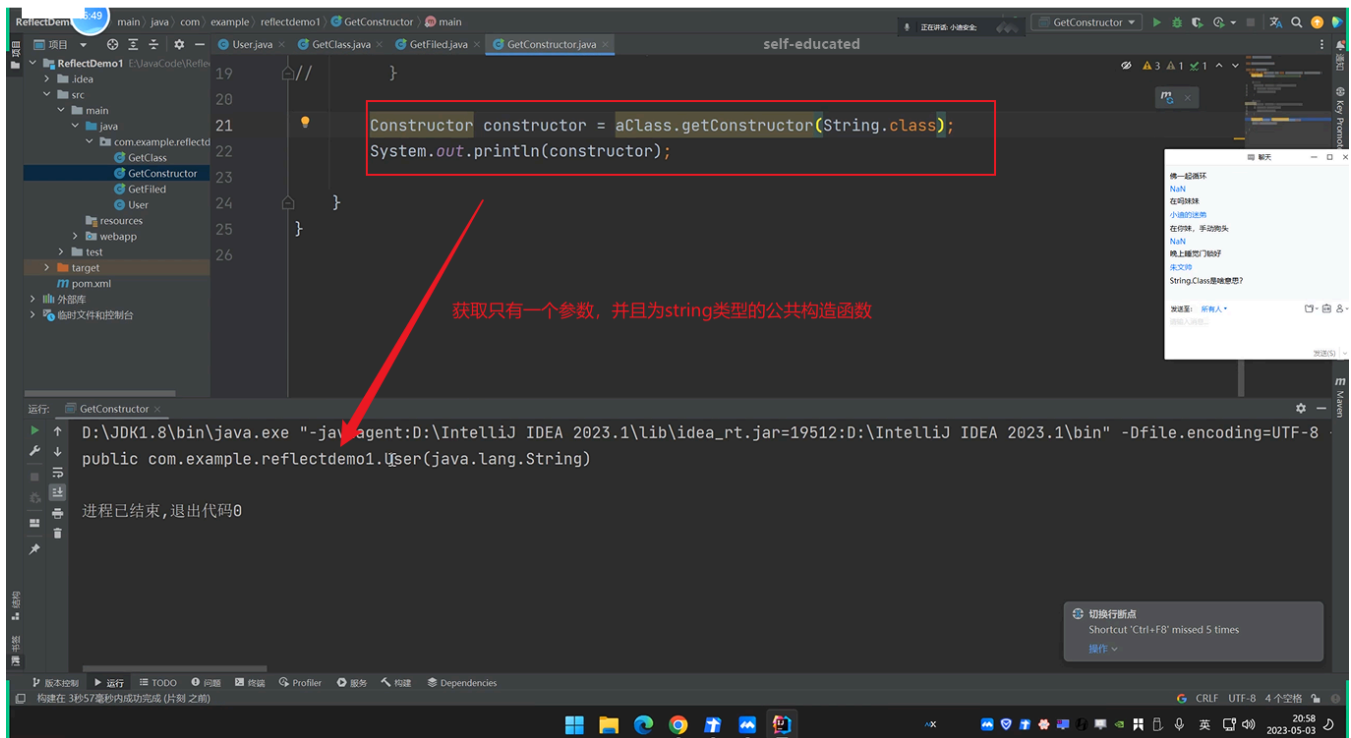
```

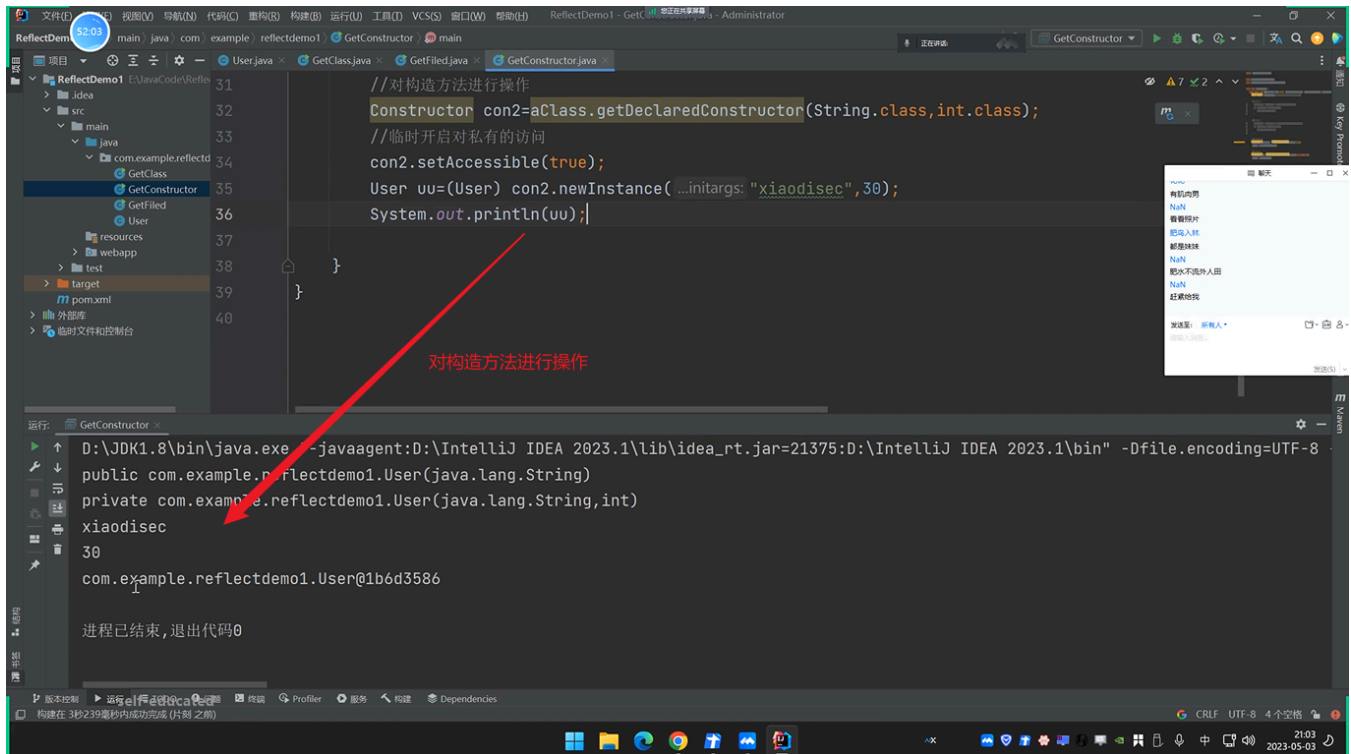
26 //      User uu=(User) con2.newInstance("xiaodi",30,"man");
27 //      System.out.println(uu);

```

(1) Constructor构造方法类获取:







2.4 Java-反射-Method成员方法类获取

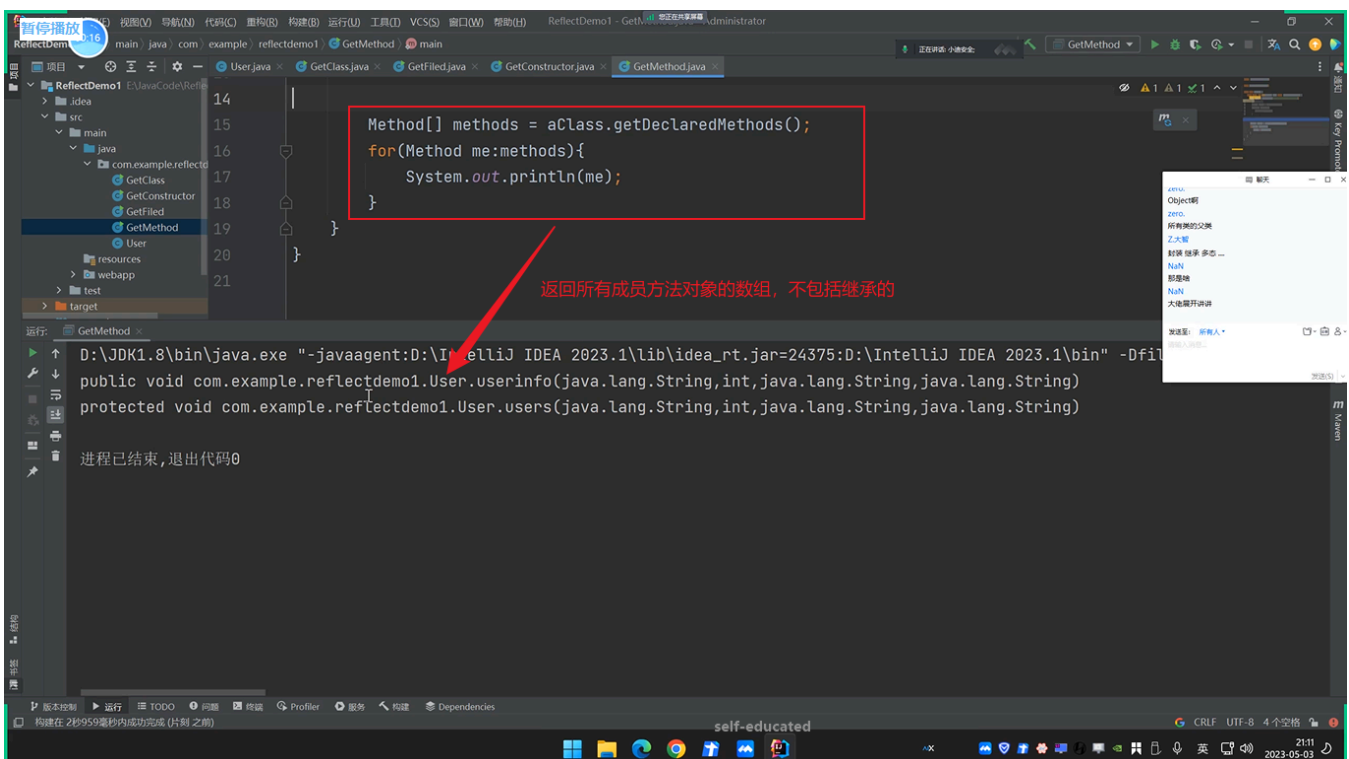
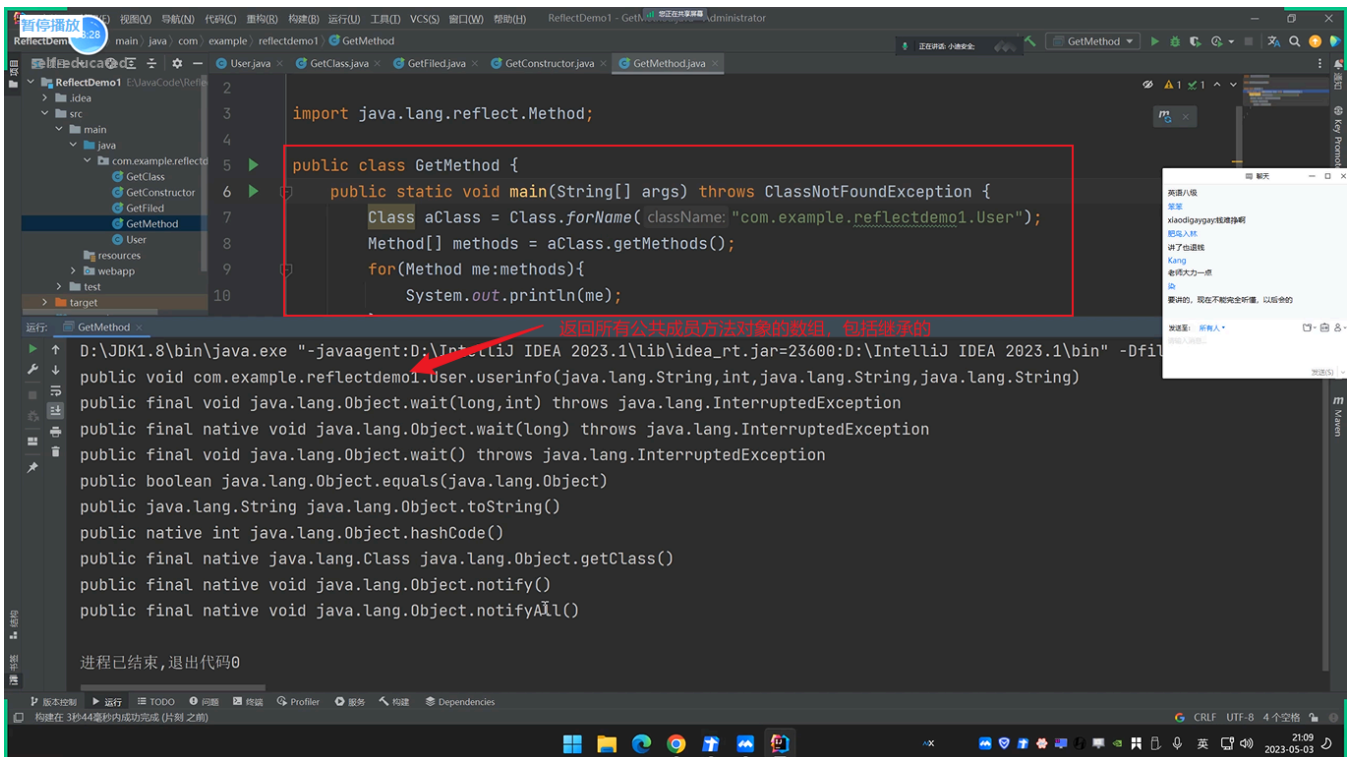
```
1 //Class aClass = Class.forName("com.example.reflectdemo.User");
2
3 //返回所有公共成员方法对象的数组, 包括继承的
4 //      Method[] methods = aClass.getMethods();
5 //      for (Method me:methods){
6 //          System.out.println(me);
7 //      }
8
9 //返回所有成员方法对象的数组, 不包括继承的
10 //      Method[] methods = aClass.getDeclaredMethods();
11 //      for (Method me:methods){
12 //          System.out.println(me);
13 //      }
14
15 //返回单个公共成员方法对象
16 //      Method methods = aClass.getMethod("getName");
17 //      System.out.println(methods);
18
19 //      Method methods = aClass.getMethod("setName", String.class);
20 //      System.out.println(methods);
21
22 //返回单个成员方法对象
23 //      Method methods = aClass.getDeclaredMethod("UserInfo", String.class,
24 //          int.class, String.class);
25 //      System.out.println(methods);
```

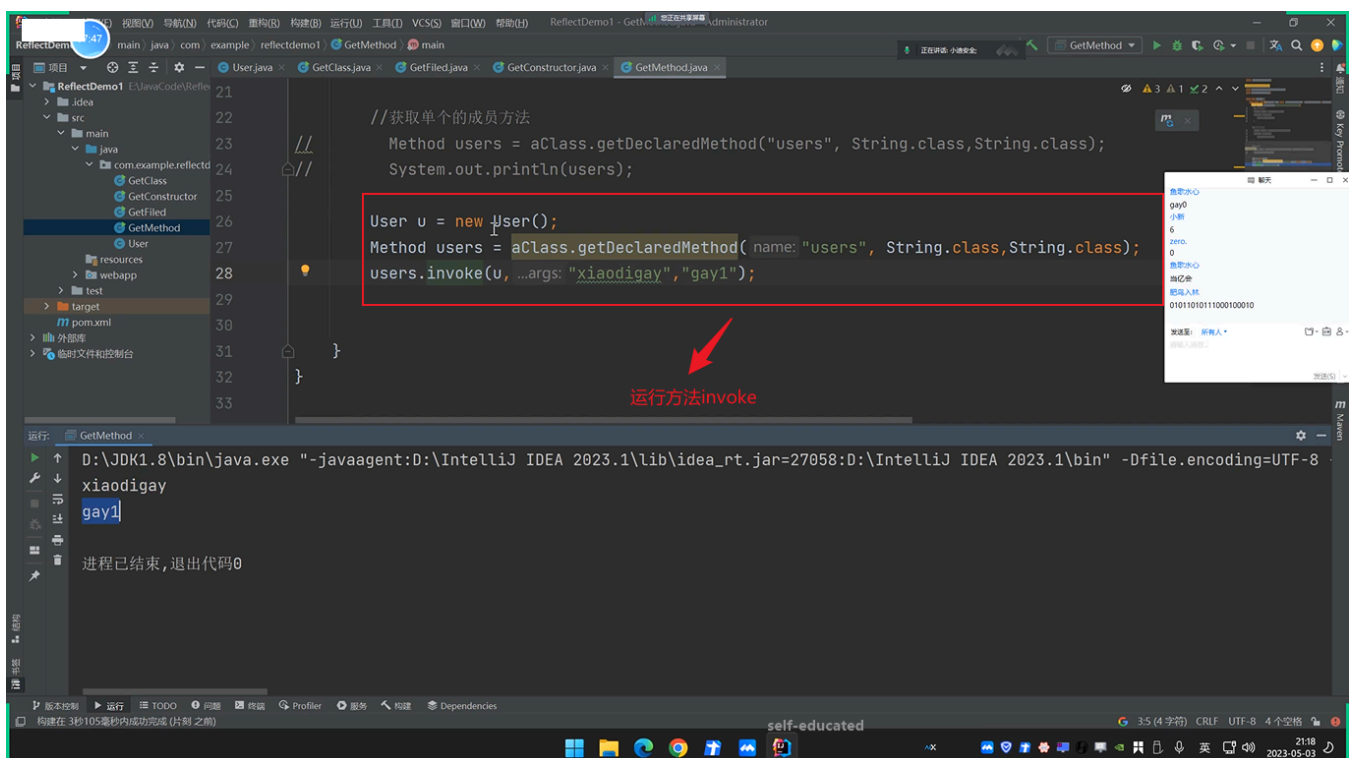
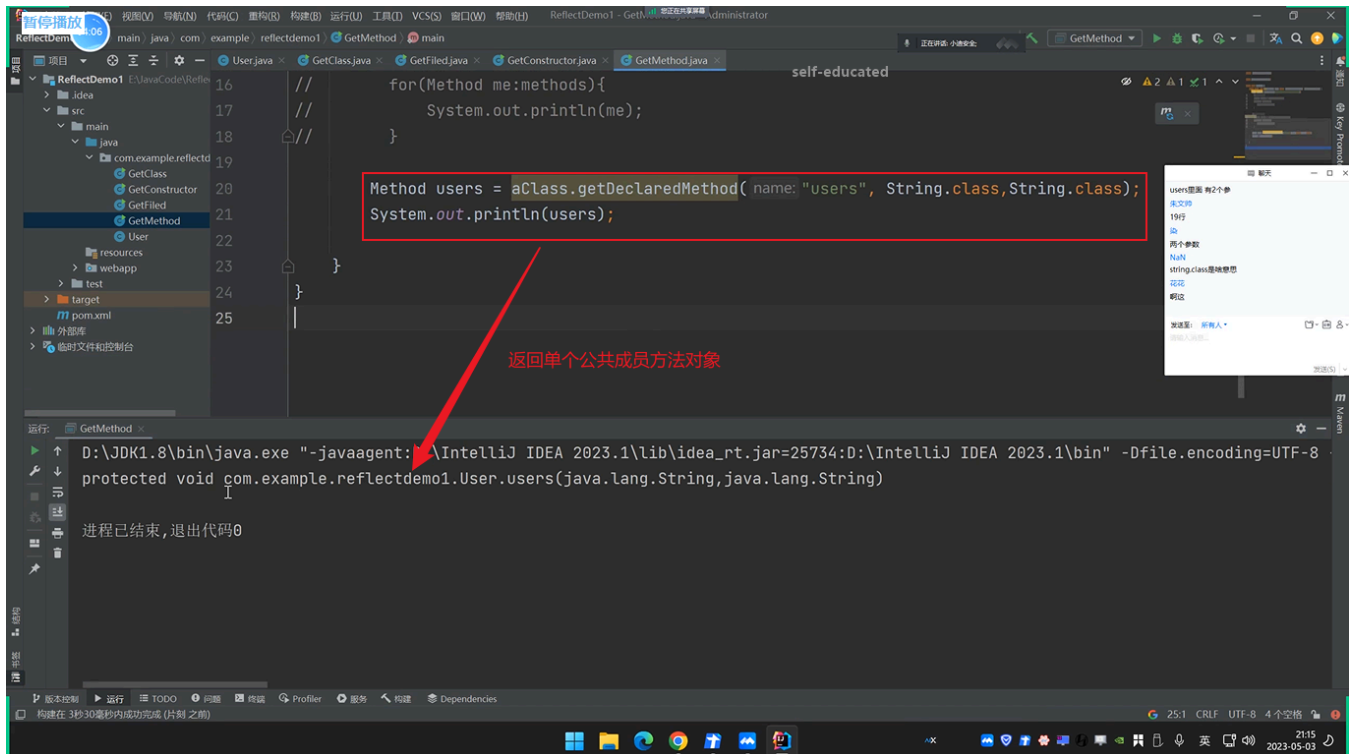
```

26 //运行方法invoke
27 //      Method methods = aClass.getDeclaredMethod("UserInfo", String.class,
28 //      int.class, String.class);
29 //      User u = new User();
30 //      //私有需要开启临时
31 //      methods.setAccessible(true);
32 //      methods.invoke(u,"xiaodi",18,"man");

```

(1) Method成员方法类获取：





2.5 Java-反射-不安全命令执行&反序列化链构造

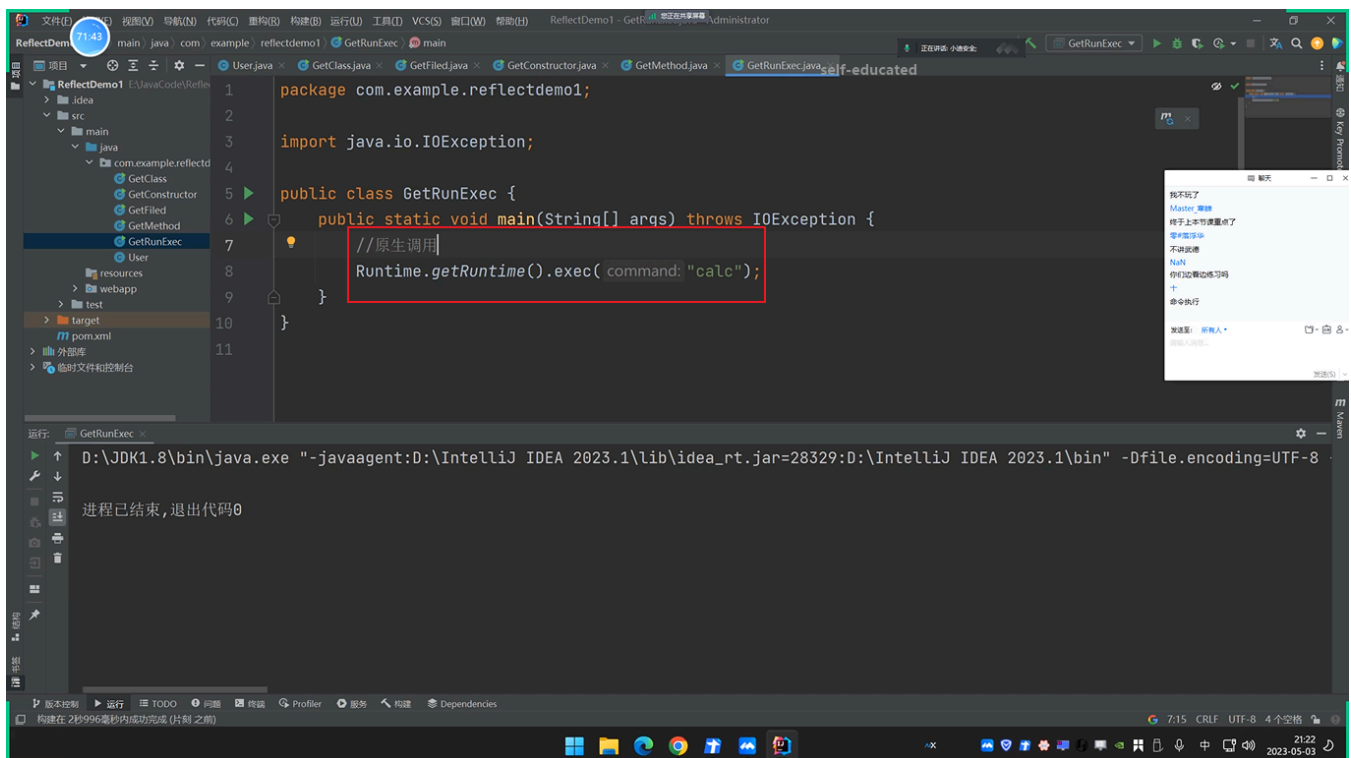
- 1 1、反射实现-命令执行
- 2 -原型:
- 3 `Runtime.getRuntime().exec("calc");`
- 4 -反射:
- 5 `Class aClass = Class.forName("java.lang.Runtime");`
- 6 `Method[] methods = aClass.getMethods();`
- 7 `for (Method me:methods){`

```

8      System.out.println(me);
9  }
10  Method exec = aClass.getMethod("exec", String.class);
11  Method getRuntimeMethod = aClass.getMethod("getRuntime");
12  Object runtime = getRuntimeMethod.invoke(aClass);
13  exec.invoke(runtime, "calc.exe");
14
15  Class c1= Class.forName("java.lang.Runtime");
16  Constructor m = c1.getDeclaredConstructor();
17  m.setAccessible(true);
18  c1.getMethod("exec", String.class).invoke(m.newInstance(), "calc");
19
20  2、不安全的反射对象
21  指应用程序使用具有反射功能的外部输入来选择要使用的类或代码，
22  可能被攻击者利用而输入或选择不正确的类。绕过身份验证或访问控制检查
23  参考分析: https://zhuanlan.zhihu.com/p/165273855
24  利用结合: https://xz.aliyun.com/t/7031（反序列化利用链）

```

(1) 原生调用:



(2) 使用反射调用:

